

Stats200Q Book

Ryder Fried

2026-06-08

Table of contents

1	Introduction	4
2	MLE & Posterior Mean	5
2.1	Maximum Likelihood Estimation	5
2.2	Posterior Mean	6
2.3	My Dataset	6
2.4	Bias and Variance	7
3	Quick Proof & Evaluating Estimators	9
3.1	Proving $\text{Var}(\ell'(\theta)) = -\mathbb{E}[\ell''(\theta)]$	9
3.2	MSE	10
3.3	Cramer-Rao Lower Bound	12
4	Asymptotic Distribution of MLE	13
4.1	Simulating Distribution	13
4.2	Asymptotic Calculation	13
4.3	Graphing Asymptotic MLE Curve	15
5	Confidence and Credible Intervals	16
5.1	Confidence Interval	16
5.1.1	Plug-In	16
5.1.2	Direct Computation	17
5.2	Credible Interval	19
6	Simulating Confidence Intervals	21
6.1	Bootstrapping	21
6.1.1	Parametric Bootstrap	21
6.1.2	Nonparametric Bootstrap	22
6.1.3	Comparing Confidence Intervals	22
6.2	Metropolis Hastings	23
7	Hypothesis Testing	26
7.1	Hypothesis Testing	26

8	Generalized Likelihood Ratio Test	29
8.1	Generalized Likelihood Ratio Test	29
8.1.1	Simulation	31
8.1.2	Asymptotic	32
9	Bayesian Hypothesis Testing	35
9.1	Bayesian Hypothesis Test	35
10	Sufficiency	39
10.1	Sufficient Statistic	39
10.1.1	First Proof	39
10.1.2	Second Proof	41
	References	42

1 Introduction

Each chapter is specified in a separate `.qmd` file. The ordering of the chapters is specified in `_quarto.yml`. The table of contents in the sidebar is automatically generated from the `_quarto.yml` file.

To render the book, you run `quarto render` in the Terminal while you are in this directory. This will generate a new folder called `_book`, which contains the HTML files that you should upload to your web host.

You can include citations by adding the BibTex to `references.bib` and citing the item by its key. (Knuth 1984)

2 MLE & Posterior Mean

2.1 Maximum Likelihood Estimation

$$X_1, \dots, X_n \sim \text{NegBin}(r, p)$$

$$P(X = x) = \binom{x+r-1}{x} p^r (1-p)^x$$

$$L_{x_1, \dots, x_n}(p) = \prod_{i=1}^n \binom{x_i+r-1}{x_i} p^r (1-p)^{x_i}$$

$$L_{x_1, \dots, x_n}(p) = p^{rn} \prod_{i=1}^n \binom{x_i+r-1}{x_i} (1-p)^{x_i}$$

$$L_{x_1, \dots, x_n}(p) = p^{rn} \left(\prod_{i=1}^n \binom{x_i+r-1}{x_i} \right) (1-p)^{\sum_{i=1}^n x_i}$$

$$l_{x_1, \dots, x_n}(p) = rn \log(p) + \sum_{i=1}^n \log \left(\frac{(x_i+r-1)!}{x_i! (r-1)!} \right) + \left(\sum_{i=1}^n x_i \right) \log(1-p)$$

$$\frac{d}{dp} l_{x_1, \dots, x_n}(p) = \frac{d}{dp} \left(rn \log(p) + \sum_{i=1}^n \log \left(\frac{(x_i+r-1)!}{x_i! (r-1)!} \right) + \left(\sum_{i=1}^n x_i \right) \log(1-p) \right)$$

$$\frac{d}{dp} l_{x_1, \dots, x_n}(p) = \left(\frac{rn}{p} - \frac{\sum_{i=1}^n x_i}{1-p} \right)$$

$$0 = \frac{rn}{p} - \frac{\sum_{i=1}^n x_i}{1-p}$$

$$\frac{rn}{p} = \frac{\sum_{i=1}^n x_i}{1-p}$$

$$\frac{1-p}{p} = \frac{\sum_{i=1}^n x_i}{rn}$$

$$\frac{1}{p} - 1 = \frac{\bar{x}}{r}$$

$$\hat{p} = \frac{r}{r + \bar{x}}$$

2.2 Posterior Mean

With a Beta Prior. Let's set $\alpha = 1, \beta = 1$. (So this is actually just a uniform).

$$\begin{aligned} f(p|x) &= \frac{f(p)f(x_1, \dots, x_n|p)}{f(x_1, \dots, x_n)} \\ f(p|x) &= \frac{\frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} p^{\alpha-1} (1-p)^{\beta-1} \prod_{i=1}^n \binom{x_i+r-1}{x_i} p^r (1-p)^{x_i}}{f(x_1, \dots, x_n)} \\ f(p|x) &\propto p^{\alpha+nr-1} (1-p)^{\beta+\sum x_i-1} \end{aligned}$$

This results in $f(p|x) \sim \text{Beta}(\alpha + nr, \beta + \sum_{i=1}^n x_i)$.

$$\begin{aligned} \mathbb{E}[P] &= \frac{\alpha + nr}{\alpha + \beta + nr + \sum_{i=1}^n x_i} \\ \mathbb{E}[P] &= \frac{1 + nr}{2 + nr + \sum_{i=1}^n x_i} \end{aligned}$$

2.3 My Dataset

In my dataset, $\bar{x} = 0.7435, n = 807, \sum(x) = 600$.

$$\begin{aligned} \hat{p}_{MLE} &= \frac{r}{r + 0.7435} \\ \hat{p}_{posterior} &= \frac{1 + 807r}{2 + 807r + 600} \end{aligned}$$

If we assume $r = 0.5$:

$$\begin{aligned} \hat{p}_{MLE} &= \frac{0.5}{0.5 + 0.7435} = 0.402 \\ \hat{p}_{posterior} &= \frac{1 + 807(0.5)}{2 + 807(0.5) + 600} = 0.401 \end{aligned}$$

2.4 Bias and Variance

Let's say true p is 0.4.

After 10,000 simulations, the bias and variance of our estimators is:

```
n_obs <- 807
true_size <- 0.5 #r
true_p <- 0.4

sims <- replicate(10000, {
  x <- rnbino(n = n_obs, size = true_size, prob = true_p)
  x_sum <- sum(x)
  x_avg = mean(x)
  p.mle <- true_size / (true_size + x_avg)

  p.mod <- (1 + n_obs * true_size) / (2 + n_obs * true_size + x_sum)

  c(mle = p.mle, mod = p.mod)
})

sims <- t(sims)

bias_mle <- mean(sims[, "mle"]) - true_p
bias_mod <- mean(sims[, "mod"]) - true_p

var_mle <- var(sims[, "mle"])
var_mod <- var(sims[, "mod"])

results <- data.frame(
  Metric = c("Bias", "Variance"),
  MLE = c(bias_mle, var_mle),
  Posterior_Mean = c(bias_mod, var_mod)
)

results
```

$\hat{p}_{MLE}$ has positive bias while $\hat{p}_{PM}$ is biased toward the prior distribution.

$\hat{p}_{PM}$ has less variance. This is more apparent if you make `n_obs` smaller in the code.

Here is the histogram for $\hat{p}_{MLE}$.

```
hist_mle <- hist(sims[, "mle"],
  breaks = 30,
  main = expression("Histogram of " * hat(p)[MLE]),
  xlab = expression(hat(p)[MLE])
)
```

Here is the histogram for $\hat{p}_{PM}$.

```
hist_mod <- hist(sims[, "mod"],
  breaks = 30,
  main = expression("Histogram of " * hat(p)[PM]),
  xlab = expression(hat(p)[PM])
)
```


3 Quick Proof & Evaluating Estimators

3.1 Proving $\text{Var}(\ell'(\theta)) = -\mathbb{E}[\ell''(\theta)]$

For any PDF $f_\theta(x)$, the log-likelihood of x can be written as $\ell_x(\theta)$.

The Score is defined as $\mathbb{E}[\ell'_\theta(x)]$.

$$\mathbb{E}[\ell'_x(\theta)] = \int_{-\infty}^{\infty} \ell'_x(\theta) f_\theta(x) dx$$

Now, let's solve for $\ell'_x(\theta)$:

$$\begin{aligned} L_x(\theta) &= f_\theta(x) \\ \ell_x(\theta) &= \log(f_\theta(x)) \\ \frac{d}{d\theta} \ell_x(\theta) &= \frac{\frac{d}{d\theta} f_\theta(x)}{f_\theta(x)} \\ \ell'_x(\theta) &= \frac{\frac{d}{d\theta} f_\theta(x)}{f_\theta(x)} \end{aligned}$$

Subbing this into our expectation:

$$\begin{aligned} \mathbb{E}[\ell'_x(\theta)] &= \int_{-\infty}^{\infty} \frac{\frac{d}{d\theta} f_\theta(x)}{f_\theta(x)} f_\theta(x) dx \\ &= \int_{-\infty}^{\infty} \frac{d}{d\theta} f_\theta(x) dx \\ &= \frac{d}{d\theta} \int_{-\infty}^{\infty} f_\theta(x) dx \\ &= \frac{d}{d\theta} (1) = 0 \end{aligned}$$

Now, solving for $\text{Var}(\ell'(\theta))$. We know: $\text{Var}(\ell'(\theta)) = \mathbb{E}[(\ell'(\theta))^2] - (\mathbb{E}[\ell'(\theta)])^2$.

Since $\mathbb{E}[\ell'_x(\theta)] = 0$:

$$\text{Var}(\ell'(\theta)) = \mathbb{E}[(\ell'(\theta))^2] = \int_{-\infty}^{\infty} (\ell'_x(\theta))^2 f_{\theta}(x) dx$$

Next, we relate $(\ell'_x(\theta))^2$ to the second derivative:

$$\begin{aligned} \ell''_x(\theta) &= \frac{d}{d\theta} \left[\frac{\frac{d}{d\theta} f_{\theta}(x)}{f_{\theta}(x)} \right] = \frac{f_{\theta}(x) \frac{d^2}{d\theta^2} f_{\theta}(x) - (\frac{d}{d\theta} f_{\theta}(x))^2}{(f_{\theta}(x))^2} \\ \ell''_x(\theta) &= \frac{\frac{d^2}{d\theta^2} f_{\theta}(x)}{f_{\theta}(x)} - \left(\frac{\frac{d}{d\theta} f_{\theta}(x)}{f_{\theta}(x)} \right)^2 \\ \ell''_x(\theta) &= \frac{\frac{d^2}{d\theta^2} f_{\theta}(x)}{f_{\theta}(x)} - (\ell'_x(\theta))^2 \end{aligned}$$

Rearranging gives $(\ell'_x(\theta))^2 = \frac{\frac{d^2}{d\theta^2} f_{\theta}(x)}{f_{\theta}(x)} - \ell''_x(\theta)$. Integrating this against the PDF:

$$\begin{aligned} \text{Var}(\ell'(\theta)) &= \int_{-\infty}^{\infty} f_{\theta}(x) \left(\frac{\frac{d^2}{d\theta^2} f_{\theta}(x)}{f_{\theta}(x)} - \ell''_x(\theta) \right) dx \\ &= \int_{-\infty}^{\infty} \frac{d^2}{d\theta^2} f_{\theta}(x) - \ell''_x(\theta) f_{\theta}(x) dx \\ &= \int_{-\infty}^{\infty} \frac{d^2}{d\theta^2} f_{\theta}(x) dx - \int_{-\infty}^{\infty} \ell''_x(\theta) f_{\theta}(x) dx \\ &= \frac{d^2}{d\theta^2} \int_{-\infty}^{\infty} f_{\theta}(x) dx - \mathbb{E}[\ell''_x(\theta)] \\ &= \frac{d^2}{d\theta^2} (1) - \mathbb{E}[\ell''_x(\theta)] \\ &= -\mathbb{E}[\ell''_x(\theta)] \end{aligned}$$

3.2 MSE

In PSET 1, I derived that if $X \sim \text{NegBin}(r, p)$, then: $\hat{p}_{MLE} = \frac{r}{r+\bar{x}}$ and $\hat{p}_{PM} = \frac{1+nr}{2+nr+\sum x_i}$.

To show their different curves MSE for different p , below is a simulation that prints out Mean Squared Error for different p , for when $n = 50$ and $r = 0.5$.

```

n <- 50
true_size <- 0.5 #r

p_grid <- seq(0.01, 0.99, length.out = 40)
mse_results <- sapply(p_grid, function(p) {

  # Run simulations for each specific p in the grid
  temp_sims <- replicate(10000, {
    x <- rbinom(n = n, size = true_size, prob = p)
    x_avg = mean(x)
    x_sum = sum(x)

    p.mle <- true_size / (true_size + x_avg)
    p.mod <- (1 + n * true_size) / (2 + n * true_size + x_sum)

    c(mle = p.mle, mod = p.mod)
  })

  # Calculate MSE for this specific p
  mse_mle_p <- mean((temp_sims["mle",] - p)^2)
  mse_mod_p <- mean((temp_sims["mod",] - p)^2)

  c(mse_mle = mse_mle_p, mse_mod = mse_mod_p)
})

# Plotting the curve
plot(p_grid, mse_results["mse_mle",], type = "l", col = "blue", lwd = 2,
     main = "MSE for Different Estimators",
     xlab = "True p",
     ylab = "MSE",
     cex.main = 0.9,
     cex.lab = 0.8,
     cex.axis = 0.7)

lines(p_grid, mse_results["mse_mod",], col = "red", lwd = 2, lty = 2)
legend("topright",
      legend = c("MLE", "Posterior Mean"),
      col = c("blue", "red"),
      lty = 1:2,
      cex = 0.6)

```

3.3 Cramer-Rao Lower Bound

The CRLB for unbiased estimators is $\frac{1}{-\mathbb{E}[\ell''(\theta)]}$. For $X \sim \text{NegBin}(r, p)$:

$$\begin{aligned}\mathbb{E}\left[\frac{d^2}{dp^2}\ell(p)\right] &= -\left(\frac{rn}{p^2} + \frac{\mathbb{E}[\sum x]}{(1-p)^2}\right) \\ &= -\left(\frac{rn}{p^2} + \frac{rn(1-p)}{p(1-p)^2}\right) \\ &= -\frac{rn}{p^2(1-p)}\end{aligned}$$

The lowest possible variance for an unbiased estimator is $\frac{p^2(1-p)}{rn}$.

However, in Week 1, we established from simulation that $\hat{p}_{MLE}$ is positively biased. Additionally, in class we learned that the $\hat{p}_{PM}$ is always biased toward the prior. Since the MLE and Posterior Mean are both biased, the CRLB does not provide a strict lower bound for their variance.

4 Asymptotic Distribution of MLE

4.1 Simulating Distribution

Let's say $X_1, \dots, X_n \sim \text{NegBin}(r, p)$, where $r = 0.5$ and $p = 0.4$.

We derived in PSET 1 that $\hat{p}_{MLE} = \frac{r}{r + \bar{x}}$. Below is a histogram of $\hat{p}_{MLE}$ as n grows large, in this case to 10,000.

```
n_obs <- 10000
true_size <- 0.5 #r
true_p <- 0.4

sims <- replicate(1000, {
  x <- rnbinom(n = n_obs, size = true_size, prob = true_p)
  x_sum <- sum(x)
  x_avg = mean(x)
  p.mle <- true_size / (true_size + x_avg)

  p.mle
})

sims <- t(sims)

hist_mle <- hist(sims,
  breaks = 100,
  main = expression("Histogram of " * hat(p)[MLE]),
  xlab = expression(hat(p)[MLE])
)
```

4.2 Asymptotic Calculation

From class, we determined that the asymptotic variance of $\hat{p}_{MLE}$ is $\frac{1}{\text{Var}(\ell'_{x_1}(\theta))}$.

Taking the score from PSET 1, we have:

$$\frac{d}{dp} \ell_{x_1, \dots, x_n}(p) = \left(\frac{rn}{p} - \frac{\sum x}{1-p} \right)$$

As we only want to work with one observation now, we will set $n = 1$.

$$\ell'_{x_1}(p) = \left(\frac{r}{p} - \frac{x}{1-p} \right)$$

We established in PSET 2 that $\text{Var}(\ell'(\theta)) = -\mathbb{E}[\ell''(\theta)]$. This means that the asymptotic variance of $\hat{p}_{MLE}$ is $\frac{1}{-\mathbb{E}[\ell''(\theta)]}$. So, let's solve for $-\mathbb{E}[\ell''(\theta)]$.

$$\begin{aligned} \ell'_{x_1}(p) &= \frac{r}{p} - \frac{x}{1-p} \\ \frac{d^2}{dp^2} \ell_{x_1}(p) &= \frac{-r}{p^2} - \frac{x}{(1-p)^2} \\ -\mathbb{E}[\ell''_{x_1}(p)] &= \frac{r}{p^2} + \frac{\mathbb{E}[x]}{(1-p)^2} \\ -\mathbb{E}[\ell''_{x_1}(p)] &= \frac{r}{p^2} + \frac{\left(\frac{r(1-p)}{p} \right)}{(1-p)^2} \\ -\mathbb{E}[\ell''_{x_1}(p)] &= \frac{r}{p^2} + \frac{r}{p(1-p)} \\ -\mathbb{E}[\ell''_{x_1}(p)] &= \frac{r(1-p)}{p^2(1-p)} + \frac{rp}{p^2(1-p)} \\ -\mathbb{E}[\ell''_{x_1}(p)] &= \frac{r(1-p) + rp}{p^2(1-p)} \\ -\mathbb{E}[\ell''_{x_1}(p)] &= \frac{r(1-p+p)}{p^2(1-p)} \\ -\mathbb{E}[\ell''_{x_1}(p)] &= \frac{r}{p^2(1-p)} \end{aligned}$$

So, the asymptotic variance of $\hat{p}_{MLE}$ is $\frac{p^2(1-p)}{r}$.

Based on the CLT, which we showed in class:

$$\sqrt{n}(\hat{p}_{MLE} - p) \sim N\left(0, \frac{p^2(1-p)}{r}\right)$$

Now, isolating for $\hat{p}_{MLE}$, we have:

$$\hat{p}_{MLE} \sim N\left(p, \frac{p^2(1-p)}{nr}\right)$$

4.3 Graphing Asymptotic MLE Curve

Below is a curve with the distribution of $\hat{p}_{MLE}$ with $r = 0.5$, $p = 0.4$, and large n (10,000), behind a red curve displaying the asymptotically normal distribution of $\hat{p}_{MLE}$.

```
hist_mle <- hist(sims,
  breaks = 100,
  main = expression("Histogram of " * hat(p)[MLE]),
  xlab = expression(hat(p)[MLE]),
  freq=F
)
curve(dnorm(x, mean=true_p,
  sd= sqrt( ( (true_p ** 2) * (1 - true_p) ) / (n_obs * true_size))),
add=T, col="red", lwd=2)

legend("topright", legend = expression("Asymptotic Normal " * hat(p)[MLE]),
  col="red", lwd=2, lty = 1, cex = 0.6)
```

5 Confidence and Credible Intervals

5.1 Confidence Interval

To remember, in Homework 1, we solved that for $X_1, \dots, X_n \sim \text{NegBin}(r, p)$ $\hat{p}_{MLE} = \frac{r}{r+\bar{x}}$.

In Homework #3, we established both that

$$\sqrt{n}(\hat{p}_{MLE} - p) \sim N\left(0, \frac{p^2(1-p)}{r}\right)$$

and

$$\hat{p}_{MLE} \dot{\sim} N\left(p, \frac{p^2(1-p)}{nr}\right)$$

This means that as the number of data points approaches infinity, $\hat{p}_{MLE}$ is approximately Normal, with a mean of p and a variance of $\frac{p^2(1-p)}{nr}$.

Therefore, an approximate 95% confidence interval for p is : $\hat{p}_{MLE} \pm 2\sqrt{\frac{p^2(1-p)}{nr}}$. However, we do not know the true value of p , so we have two options in how we can proceed.

5.1.1 Plug-In

We can try plugging in $\hat{p}_{MLE}$ for p in this equation. This means that the approximate 95% confidence interval is:

$$\begin{aligned}
\hat{p} \pm 2\sqrt{\frac{\hat{p}_{MLE}^2(1-\hat{p}_{MLE})}{nr}} &= \hat{p} \pm 2\sqrt{\frac{\left(\frac{r}{r+\bar{x}}\right)^2 \left(1 - \frac{r}{r+\bar{x}}\right)}{nr}} \\
&= \hat{p} \pm 2\sqrt{\frac{\left(\frac{r}{r+\bar{x}}\right)^2 \left(\frac{r+\bar{x}}{r+\bar{x}} - \frac{r}{r+\bar{x}}\right)}{nr}} \\
&= \hat{p} \pm 2\sqrt{\frac{\left(\frac{r}{r+\bar{x}}\right)^2 \left(\frac{\bar{x}}{r+\bar{x}}\right)}{nr}} \\
&= \hat{p} \pm 2\sqrt{\frac{r^2\bar{x}}{nr(r+\bar{x})^3}} \\
&= \hat{p} \pm 2\sqrt{\frac{r\bar{x}}{n(r+\bar{x})^3}}
\end{aligned}$$

Going back to my hospital dataset:

```

dat <- read.csv("https://raw.githubusercontent.com/Ryder4real/STATS200Q_Book/main/transmissi
x <- as.vector(table(dat$infector_id)) - 1
x_bar = mean(x)
n = length(x)
print(c(x_bar, n))

```

I had $\bar{x} \approx 0.7435$, $n = 807$. Let's say $r = 0.5$.

I then have:

$$\begin{aligned}
&\hat{p} \pm 2\sqrt{\frac{r\bar{x}}{n(r+\bar{x})^3}} \\
&\frac{r}{r+\bar{x}} \pm 2\sqrt{\frac{r\bar{x}}{n(r+\bar{x})^3}} \\
&\frac{0.5}{0.5 + 0.7435} \pm 2\sqrt{\frac{(0.5)(0.7435)}{807(0.5 + 0.7435)^3}} \\
&0.4021 \pm 2\sqrt{0.00023958}
\end{aligned}$$

So, the approximate plug-in 95% confidence interval for p is $[0.3711, 0.4331]$.

5.1.2 Direct Computation

Instead of plugging in $\hat{p}_{MLE}$ for p , we can try to solve for what values of p make up the confidence interval.

We established earlier that:

$$\sqrt{n}(\hat{p}_{MLE} - p) \sim N\left(0, \frac{p^2(1-p)}{r}\right)$$

This means that: $\hat{p}_{MLE} - p \sim N\left(0, \frac{p^2(1-p)}{nr}\right)$.

Furthermore, we can say that:

$$\frac{\hat{p}_{MLE} - p}{\sqrt{\frac{p^2(1-p)}{nr}}} \sim N(0, 1)$$

As this has a standard deviation of 1, we can set up a 95% confidence interval to be:

$$-2 \leq \frac{\hat{p}_{MLE} - p}{\sqrt{\frac{p^2(1-p)}{nr}}} \leq 2$$

$$\left(\frac{\hat{p}_{MLE} - p}{\sqrt{\frac{p^2(1-p)}{nr}}}\right)^2 \leq 4$$

$$\frac{(\hat{p}_{MLE} - p)^2}{\frac{p^2(1-p)}{nr}} \leq 4$$

$$\frac{nr(\hat{p}_{MLE} - p)^2}{p^2(1-p)} \leq 4$$

$$\hat{p}_{MLE}^2 - 2\hat{p}_{MLE}p + p^2 \leq \frac{4}{nr}p^2(1-p)$$

$$\hat{p}_{MLE}^2 - 2\hat{p}_{MLE}p + p^2 - \frac{4p^2}{nr} + \frac{4p^3}{nr} \leq 0$$

$$\frac{4}{nr}p^3 + \left(1 - \frac{4}{nr}\right)p^2 - 2\hat{p}_{MLE}p + \hat{p}_{MLE}^2 \leq 0$$

This is unfortunately not cleanly solvable for p , but we can graph it. If we say that $f(p) = \frac{4}{nr}p^3 + \left(1 - \frac{4}{nr}\right)p^2 - 2\hat{p}_{MLE}p + \hat{p}_{MLE}^2$, we can look at the zeros to see the values of p at the edge of the confidence intervals.

For this sample, let's go back to the dataset I started with, where $\bar{x} = 0.7435$, $n = 807$. Let's fix $r = 0.5$.

```
r <- 0.5

p_hat <- r / (r + x_bar)

# f(p)
cubic_f <- function(p) {
  a <- 4 / (n * r)
  (a * p^3) + (1 - a) * p^2 - (2 * p_hat) * p + p_hat^2
}
```

```
# ind roots of f(p) in two window [0, p_hat] and [p_hat, 1]
lower_bound <- uniroot(cubic_f, interval = c(0, p_hat))$root
upper_bound <- uniroot(cubic_f, interval = c(p_hat, 1))$root

cat("95% Confidence Interval: [", round(lower_bound, 4), ",", round(upper_bound, 4), "]\n")
```

From the computation in the code above, we are approximately 95% confident that p is in $[0.3727, 0.4346]$

5.2 Credible Interval

In Homework 1, I solved that given a $\text{Beta}(\alpha, \beta)$ prior, the posterior distribution for p is:

$$f(p|x_1, \dots, x_n) \sim \text{Beta}(\alpha + nr, \beta + \sum_{i=1}^n x_i).$$

In Homework 1, I used a uniform prior, meaning $\alpha = 1, \beta = 1$. This means the posterior distribution given a uniform prior is:

$$f(p|x_1, \dots, x_n) \sim \text{Beta}(1 + nr, 1 + \sum_{i=1}^n x_i).$$

For our 95% credible interval, we want the interval to cover 95% of the distribution with symmetric tails.

So, we need a left bound (LB) and right bound (RB), such that:

$$\int_0^{LB} \frac{p^{(1+nr)-1}(1-p)^{(1+\sum x_i)-1}}{B(1+nr, 1+\sum x_i)} dp = 0.025$$

$$\int_{RB}^1 \frac{p^{(1+nr)-1}(1-p)^{(1+\sum x_i)-1}}{B(1+nr, 1+\sum x_i)} dp = 0.025$$

Unfortunately, we can't solve for LB and RB by hand. However, the computer can solve it for us, using the 'qbeta' function.

Again, we will use the same values from my hospital dataset, as before.

```
r <- 0.5
sum_xi <- x_bar * n
credible_range <- 0.95

# Posterior Parameters based on f(p | x) ~ Beta(1 + nr, 1 + sum x_i)
alpha_post <- 1 + (n * r)
beta_post <- 1 + sum_xi

# Calculate Tail Areas
```

```

alpha_level <- 1 - credible_range
tail_area   <- alpha_level / 2

# 4. Solve for Credible Interval Bounds
LB <- qbeta(tail_area,
            shape1 = alpha_post,
            shape2 = beta_post,
            lower.tail = TRUE)

RB <- qbeta(tail_area,
            shape1 = alpha_post,
            shape2 = beta_post,
            lower.tail = FALSE)

credible_interval <- c(LB, RB)
names(credible_interval) <- c("Left Bound", "Right Bound")

print(credible_interval)

```

From the computation in the code above, conditional on my uniform prior, the credible interval states that there is a 95% chance p is in $[0.3722, 0.4328]$.

6 Simulating Confidence Intervals

First, let's bring my hospital dataset into R.

```
dat <- read.csv('transmission_pairs_covid_v2.csv')
```

Now, the data that has a negative binomial distribution is the number of people each person infected beyond the first person (This is because all infectors in the dataset infected at least one person, so we subtract one to have a healthy amount of zero values).

```
x <- as.vector(table(dat$infector_id)) - 1
```

6.1 Bootstrapping

6.1.1 Parametric Bootstrap

If we say $X \sim \text{NegBin}(p, r)$, and we have $r = 0.5$, then as I calculated in Homework 1, $\hat{p}_{MLE} = \frac{r}{r + \bar{x}} = \frac{0.5}{0.5 + 0.7435} = 0.402$.

```
r = 0.5
n = length(x)
x_bar = sum(x) / n
p_mle = r / (r + x_bar)
```

So, using a parametric bootstrap, we can simulate:

```
p.pboot <- replicate(10000, {
  # Generate dataset
  x.pboot = rnbino(n, r, p_mle)

  # Calculate new MLE
  x.pboot.bar = sum(x.pboot) / n
  p.pboot.mle = r / (r + x.pboot.bar)
  p.pboot.mle
})
```

```

    }
  )

hist(p.pboot)

```

```
quantile(p.pboot, c(.025, .975))
```

So, our approximate 95% confidence interval for p using Parametric Bootstrapping is about $[0.3738, 0.4336]$, depending on the simulation.

6.1.2 Nonparametric Bootstrap

Here, we will just re-sample from our sample.

So, using a nonparametric bootstrap, we can simulate:

```

p.npboot <- replicate(10000, {

  # Generate new sample (with replacement) from our above sample
  x.npboot <- sample(x, size=n, replace=TRUE)

  # Calculate new MLE
  x.npboot.bar = sum(x.npboot) / n
  p.npboot.mle = r / (r + x.npboot.bar)
  p.npboot.mle
}
)

hist(p.npboot)

```

```
quantile(p.npboot, c(.025, .975))
```

So, our approximate 95% confidence interval for p using Nonparametric Bootstrapping is about $[0.3714, 0.4360]$, depending on the simulation.

6.1.3 Comparing Confidence Intervals

Combining last week and this week, I have now computed four different 95% confidence intervals for p .

They are listed below:

1. Using CLT with $\hat{p}$ plug in for p in $I(p)$: [0.3711, 0.4331]
2. Using CLT with direct computation for p in $I(p)$: [0.3727, 0.4346]
3. Parametric Bootstrap: [0.3738, 0.4336]
4. Nonparametric Bootstrap: [0.3714, 0.4360]

Computing for the **range** of the 95% confidence interval, we have:

Using CLT with p_{hat} plug in for p in $I(p)$: 0.0620

Using CLT with direct computation for p in $I(p)$: 0.0619

Parametric Bootstrap: 0.0598

Nonparametric Bootstrap: 0.0646

The parametric bootstrap 95% confidence interval has the smallest range, while the nonparametric bootstrap has the largest range.

6.2 Metropolis Hastings

In Homework 1, I calculated a Bayesian posterior mean for my data using a Beta(1, 1) (Uniform) prior. I solved that:

$$f(p|x) \propto p^{\alpha+nr-1}(1-p)^{\beta+\sum x_i-1}.$$

For my uniform prior, this means that:

$$f(p|x) \propto p^{1+nr-1}(1-p)^{1+\sum x_i-1}$$

$$f(p|x) \propto p^{nr}(1-p)^{\sum x_i}.$$

Instead of recognizing the beta distribution, let's instead run the Metropolis Hastings Algorithm.

To avoid numerical underflow, we will compute $e^{\log(f(p|x))}$.

$$f(p|x) \propto e^{nr \log(p) + \sum x_i \log(1-p) + C}.$$

```
# This function calculates the posterior probability (without accounting for the constant term)
# To avoid numerical underflow, we will first calculate the log, and then do e ^ (log(prob))
post <- function(p) {
  # calculate log posterior (for numerical stability)
  log.p.post <- log(p) * (n * r) + sum(x) * log(1-p)
  exp(log.p.post)
}

# Start with an arbitrary value of p.
```

```

p.post <- c(0.5)

# Run Metropolis-Hastings for 15000 steps
for(i in 1:15000) {

  p.curr <- p.post[length(p.post)]

  # Propose a new value of p. Let's choose from uniform(0,1). Let's call this q.
  p.prop <- runif(1)

  # calculate probability of choosing new_p.
  # This is (post(new) * q(old)) / (post(old) ( q(new)))
  prob <- min(1,
    (post(p.prop) * dunif(p.curr, 0, 1) /
    (post(p.curr) * dunif(p.prop, 0, 1)))
  )

  # add new p value to list
  p.new <- sample(c(p.prop, p.curr),
    size=1, prob=c(prob, 1-prob))
  p.post <- c(p.post, p.new)
}

# Keep the latter 10,000 samples
late.p.post <- p.post[5001:15000]

# We can estimate the posterior mean by taking the mean of these samples.
mean(late.p.post)

```

Here, my Metropolis-Hastings posterior mean is around 0.401, depending on the simulation.

In Homework 1, instead of using a simulation, I recognized the $\text{Beta}(1 + nr, 1 + \sum_{i=1}^n x_i)$ distribution. In that situation, my posterior mean was 0.401.

Now, let's look at the distribution of my Metropolis-Hastings $\hat{p}$.

```
hist(late.p.post)
```

```
quantile(late.p.post, c(.025, .975))
```

Here, my Metropolis-Hastings approximate 95% credible interval is around [0.3701, 0.4283], depending on the simulation.

When I did this from the Beta function in Homework 4 using the 'qbeta' function, my 95% credible interval was [0.3722, 0.4328].

Here are some statistics about these 95% credible intervals:

Range:

Metropolis-Hastings: 0.0582

Direct computation of the posterior distribution: 0.0606

So, my approximate M-H 95% credible interval has a slightly smaller spread.

7 Hypothesis Testing

First, let's bring my hospital dataset into R.

```
dat <- read.csv('transmission_pairs_covid_v2.csv')
x <- as.vector(table(dat$infector_id)) - 1
```

7.1 Hypothesis Testing

In my hospital dataset, the data that has a negative binomial distribution is the number of people each person infected beyond the first person (This is because all infectors in the dataset infected at least one person, so we subtract one to have a healthy amount of zero values). Google says that for the Flu, the average person with the disease spreads it to 1-2 people. As our negative binomial distribution only counts the number of additional people each person infects after the first, then setting $E[X] = 0.5$ feels fair. In other words, for the flu, it reasonable to say the average person in a hospital with a disease infects 0.5 extra people. That would mean that $E[X] = \frac{r(1-p)}{p} = 0.5$.

Isolating for p , we get:

$$\begin{aligned} E[X] &= \frac{r(1-p)}{p} \\ \frac{p}{1-p} &= \frac{r}{E[X]} \\ \frac{1-p}{p} &= \frac{E[X]}{r} \\ \frac{1}{p} - 1 &= \frac{E[X]}{r} \\ \frac{1}{p} &= \frac{E[X] + r}{r} \\ p &= \frac{r}{E[X] + r} \end{aligned}$$

So, we set E to be 0.5. Let's say for the sake of this example that we set r to be 0.75. In this scenario, we get $p = \frac{0.75}{0.5+0.75} = 0.6$.

Let's say our null hypothesis is $H_0 : p = 0.6$. Now, we are curious if maybe COVID is more infectious than the flu (not considering other considerations like how my dataset is limited to people only already at the hospital). This would mean that $E[X]$ is greater than 0.5, and as $E[X] = \frac{r(1-p)}{p}$, a smaller p leads to a larger $E[X]$.

This means that:

$$H_0 : p = 0.6$$

$$H_A : p < 0.6$$

As we learned in class, to figure out the best X values to reject when $p < 0.6$, all we need to do is choose one $p < 0.6$, and the same will hold true for rest. Let's proceed with $p = 0.3$.

Again, let's proceed with $r = 0.75$.

NegBin(p, r) follows the distribution $\binom{x+r-1}{x} p^r (1-p)^x$:

So, our H_0 gives us NegBin($p = 0.6, r$), which follows the distribution: $f_0(x) = \prod \binom{x+r-1}{x} 0.6^r (1-0.6)^x$.

Our H_A gives us NegBin($p = 0.3, r$), which follows the distribution: $f_A(x) = \prod \binom{x+r-1}{x} 0.3^r (1-0.3)^x$.

We want to maximize $\frac{f_A(x)}{f_0(x)} = \prod \frac{\binom{x+r-1}{x} 0.3^r (1-0.3)^x}{\binom{x+r-1}{x} 0.6^r (1-0.6)^x}$, as these are the X values that we maximize our $\frac{\text{Power}}{\alpha}$. In other words, when this value ratio is maximized, we have the best ratio for decreasing the odds of a Type II Error while still protecting against Type I Errors.

So, let's maximize:

$$\begin{aligned} \frac{f_A(x)}{f_0(x)} &= \frac{\prod_{i=1}^n \binom{x_i+r-1}{x_i} 0.3^r (1-0.3)^{x_i}}{\prod_{i=1}^n \binom{x_i+r-1}{x_i} 0.6^r (1-0.6)^{x_i}} \\ \frac{f_A(x)}{f_0(x)} &= \frac{\left(\prod_{i=1}^n \binom{x_i+r-1}{x_i} \right) 0.3^{nr} (1-0.3)^{\sum x_i}}{\left(\prod_{i=1}^n \binom{x_i+r-1}{x_i} \right) 0.6^{nr} (1-0.6)^{\sum x_i}} \\ &= \frac{0.3^{nr} (1-0.3)^{\sum_{i=1}^n x_i}}{0.6^{nr} (1-0.6)^{\sum_{i=1}^n x_i}} \\ &\propto \frac{0.7^{\sum_{i=1}^n x_i}}{0.4^{\sum_{i=1}^n x_i}} \\ &= \left(\frac{0.7}{0.4} \right)^{\sum_{i=1}^n x_i} \end{aligned}$$

To maximize, $\left(\frac{0.7}{0.4} \right)^{\sum_{i=1}^n x_i}$, we want to reject when $\sum_{i=1}^n x_i$ is as large as possible.

So, we want: $P(\sum_{i=1}^n x_i \geq c) = 0.05$.

The sum of many $\text{NegBin}(p, r)$ random variables is $\text{NegBin}(p, n * r)$, My COVID hospital dataset had 807 patients, so $n = 807$.

```
n = length(x)
n
```

So, we would use the `qnbinom` R function to set up an sum from c to ∞ over the H_0 pdf that covers the upper 5% (at most) of the distribution.

```
r = 0.75
alpha = 0.05
p = 0.6
reject_val = qnbinom(1-alpha, r * n, p)
reject_val
```

We then check to see how much of the density this sum has (I subtract by 1 because we want the density above whichever value is inputted into the `pnbinom` function):

```
1 - pnbinom(reject_val - 1, n * r, p)
```

We are above 0.05. So, we need to move our rejection bound up one value, to take up less of the density.

```
new_reject_val = reject_val + 1
1 - pnbinom(new_reject_val - 1, n * r, p)
```

α is now below 0.05!

```
new_reject_val
```

So, $c = 448$

So, we reject if $H_0 \sum x_i \geq 448$.

Now, let's see how often we would reject in my dataset.

```
observed_sum <- sum(x)
observed_sum
```

As $600 > c$ (which was 448), for my COVID hospital dataset, we reject H_0 . This implies that we accept the hypothesis that COVID is more contagious than the flu (though this dataset is not perfect as it only consists of people already in the hospital who infected at least 1 person, so I do not want to extrapolate too much).

8 Generalized Likelihood Ratio Test

First, let's bring my hospital dataset into R.

```
dat <- read.csv('/Users/ryderfried/Documents/2025-2026/Spring/STATS200Q/transmission_pairs_c
```

```
dat <- read.csv('transmission_pairs_covid_v2.csv')
x <- as.vector(table(dat$infector_id)) - 1
```

8.1 Generalized Likelihood Ratio Test

I mention this in Homework 6, but as a reminder:

In my hospital dataset, the data that has a negative binomial distribution is the number of people each person infected beyond the first person (This is because all infectors in the dataset infected at least one person, so we subtract one to have a healthy amount of zero values). Google says that for the Flu, the average person with the disease spreads it to 1-2 people. As our negative binomial distribution only counts the number of additional people each person infects after the first, then setting $E[X] = 0.5$ feels fair. In other words, for the flu, it reasonable to say the average person in a hospital with a disease infects 0.5 extra people. That would mean that $\mathbb{E}[X] = \frac{r(1-p)}{p} = 0.5$.

I isolated for p in Homework 6, and got that $p = 0.6$:

Let's say our null hypothesis is $H_0: p = 0.6$. Now, we are curious if maybe COVID has a different degree of infectiousness than the flu (not considering other considerations like how my dataset is limited to people only already at the hospital). This would mean that $E[X] \neq 0.5$, and $p \neq 0.6$.

So, this gives us:

$$H_0: p = 0.6$$

$$H_A: p \neq 0.6$$

As we learned in class, for this type of hypothesis test, we try to maximize the ratio:

$$\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}$$

We can rewrite this as:

$$\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)} = \frac{\prod_{i=1}^n \binom{x_i+r-1}{x_i} p^r (1-p)^{x_i}}{\prod_{i=1}^n \binom{x_i+r-1}{x_i} (p_0)^r (1-p_0)^{x_i}}$$

We know that for the numerator, the p that gives the greatest likelihood is $\hat{p}_{MLE}$. In Homework 1, I calculate the $\hat{p}_{MLE}$ for multiple samples of the Negative Binomial Distribution is $\frac{r}{r+\bar{x}}$.

$$\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)} = \frac{\prod_{i=1}^n \binom{x_i+r-1}{x_i} \left(\frac{r}{r+\bar{x}}\right)^r \left(1 - \frac{r}{r+\bar{x}}\right)^{x_i}}{\prod_{i=1}^n \binom{x_i+r-1}{x_i} (p_0)^r (1-p_0)^{x_i}}$$

We can simplify this to:

$$\begin{aligned} \frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)} &= \frac{\prod_{i=1}^n \binom{x_i+r-1}{x_i} \left(\frac{r}{r+\bar{x}}\right)^r \left(1 - \frac{r}{r+\bar{x}}\right)^{x_i}}{\prod_{i=1}^n \binom{x_i+r-1}{x_i} (p_0)^r (1-p_0)^{x_i}} \\ \frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)} &= \frac{\left(\frac{r}{r+\bar{x}}\right)^{nr} \left(1 - \frac{r}{r+\bar{x}}\right)^{\sum_{i=1}^n x_i}}{(p_0)^{nr} (1-p_0)^{\sum_{i=1}^n x_i}} \end{aligned}$$

We want to reject when $\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)} \geq c$, where:

$$P(\text{test_statistic} \geq c) = \alpha = 0.05$$

.

However, this does not simplify to a distribution I recognize, so I cannot solve for c .

Still, we want to maximize this function. As $\log()$ is monotonic, we can take the logarithm of the fraction.

$$\begin{aligned} \log\left(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}\right) &= \log\left(\frac{\left(\frac{r}{r+\bar{x}}\right)^{nr} \left(1 - \frac{r}{r+\bar{x}}\right)^{\sum_{i=1}^n x_i}}{(p_0)^{nr} (1-p_0)^{\sum_{i=1}^n x_i}}\right) \\ \log\left(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}\right) &= nr \log\left(\frac{r}{r+\bar{x}}\right) + \sum_{i=1}^n x_i \log\left(1 - \frac{r}{r+\bar{x}}\right) - \left(nr \log(p_0) + \sum_{i=1}^n x_i \log(1-p_0)\right) \\ \log\left(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}\right) &= nr \log\left(\frac{r}{r+\bar{x}}\right) + \sum_{i=1}^n x_i \log\left(\frac{1 - \frac{r}{r+\bar{x}}}{1-p_0}\right) \\ \log\left(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}\right) &= nr \log\left(\frac{r}{r+\bar{x}}\right) + \sum_{i=1}^n x_i \log\left(\frac{1 - \frac{r}{r+\bar{x}}}{1-p_0}\right) \end{aligned}$$

Once again, this also does not simplify to a distribution I recognize. So, we are left to either simulate or take the asymptotic distribution.

8.1.1 Simulation

Let's simulate our many times under the null hypothesis, and calculate $\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}$ each time. For brevity, I will refer to $\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}$ as Λ . Then, as we want to keep $\alpha \leq 0.05$, we would reject if Λ for our data is greater than the 95th percentile of the simulated Λ s.

$$\text{Remember: } \Lambda = \frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)} = \frac{(\frac{r}{r+x})^{nr} (1 - \frac{r}{r+x})^{\sum_{i=1}^n x_i}}{(p_0)^{nr} (1-p_0)^{\sum_{i=1}^n x_i}}.$$

However, to prevent underflow, I will take the logarithm.

$$\log\left(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}\right) = nr \log\left(\frac{r}{r+x}\right) + \sum_{i=1}^n x_i \log\left(\frac{1 - \frac{r}{r+x}}{1-p_0}\right)$$

```
n = length(x)
r = 0.75
p_0 = 0.6

sims <- replicate(10000, {
  data <- rbinom(n = n, size = r, prob = p_0)

  data_bar = mean(data)
  data_sum = sum(data)

  p_mle = r / (r + data_bar)

  first_term = n * r * log(p_mle / p_0)
  second_term = data_sum * log((1 - p_mle) / (1 - p_0))

  log_lambda = first_term + second_term
  log_lambda
})
quantile(sims, 0.95)
```

So, our test statistic is $\log\left(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}\right)$ and our c is about 1.933, depending on the simulation. So, if Λ for my data is above 1.933, we reject H_0 .

So, let's calculate $\log(\Lambda)$, while setting $r = 0.75$.

```
n = length(x)
r = 0.75
p_0 = 0.6

x_bar = mean(x)
```

```

x_sum = sum(x)

p_mle = r / (r + x_bar)

first_term = n * r * log(p_mle / p_0)
second_term = x_sum * log((1 - p_mle) / (1 - p_0))

log_lambda = first_term + second_term
log_lambda

```

As $\log(\Lambda)$ is 23.55, which is $> c$, we reject H_0 .

8.1.2 Asymptotic

Remember, we want to reject when:

$$2 \log\left(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}\right) \geq c$$

We learned in class that under Wilks' Theorem, for large n :

$$2 \log\left(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}\right) \sim \chi_1^2.$$

So, this would make c the 95% percentile of the chi-squared distribution.

So, let's figure out our c :

We can run this in code:

```

alpha = 0.05
p = 0.6
c = qchisq(1-alpha, 1)
c

```

So, $c = 3.841459$

Now, let's check our data. Let's assume $r = 0.75$. Remember that Wilks' Theorem states that for large n : $2 \log\left(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}\right) \sim \chi_1^2$.

For our negative binomial: $\log\left(\frac{\max_p \mathcal{L}(p)}{\mathcal{L}(p_0)}\right) = nr \log\left(\frac{r}{r+x}\right) + \sum_{i=1}^n x_i \log\left(\frac{1-\frac{r}{r+x}}{1-p_0}\right)$

Let's proceed with $r = 0.75$.


```

n = length(x)
r = 0.75
x_sum = sum(x)
x_bar = mean(x)
p_0 = 0.6
p_mle = r / (r + x_bar)
log_part = n * r * log(p_mle / p_0) + x_sum * log((1 - p_mle) / (1 - p_0))
test_stat = 2 * log_part
test_stat

```

If I just plug in the PDFs, I get the same value:

```

log_L_mle <- dnbinom(x_sum, size = n * r, prob = p_mle, log = TRUE)
log_L_null <- dnbinom(x_sum, size = n * r, prob = p_0, log = TRUE)

test_stat_2 = 2 * (log_L_mle - log_L_null)
test_stat_2

```

As $47.1 > c$ (which was 3.841459), for my COVID hospital dataset, we reject H_0 . This implies that we reject the hypothesis that COVID has the same contagious level as the flu (though this dataset is not perfect as it only consists of people already in the hospital who infected at least 1 person, so I do not want to extrapolate too much). Another way to say this is that we accept the hypothesis that COVID does not have the same contagious level as the flu.

A power curve would be:

```

p_alt <- seq(0.45, 0.75, by = 0.001)

lambda <- 2 * n * r * (log(p_alt / p_0) + ((1 - p_alt) / p_alt) * log((1 - p_alt) / (1 - p_0)))

lambda[p_alt == p_0] <- 0

power_curve <- pchisq(c, df = 1, ncp = lambda, lower.tail = FALSE)

# --- 4. Plot the Power Curve ---
plot(p_alt, power_curve, type = "l", col = "#2c3e50", lwd = 3,
     xlab = expression(textstyle("p")),
     ylab = "Power",
     main = "GLR Power Curve",
     ylim = c(0, 1), panel.first = grid(lty = "dotted", col = "gray70"))

# Add visual anchors for interpretation

```

```
abline(v = p_0, col = "#e74c3c", lty = 2, lwd = 2)    # Null Hypothesis baseline

# Add a legend for clarity
legend("bottomright",
      legend = c("Power Curve", "Null Hypothesis (p = 0.6)"),
      col = c("#2c3e50", "#e74c3c", "#27ae60"),
      cex=0.6,
      lty = c(1, 2, 3), lwd = c(3, 2, 2), bg = "white")
```

9 Bayesian Hypothesis Testing

First, let's bring my hospital dataset into R.

```
dat <- read.csv('transmission_pairs_covid_v2.csv')
x <- as.vector(table(dat$infector_id)) - 1
```

9.1 Bayesian Hypothesis Test

I mention this in Homework 6, but as a reminder:

In my hospital dataset, the data that has a negative binomial distribution is the number of people each person infected beyond the first person (This is because all infectors in the dataset infected at least one person, so we subtract one to have a healthy amount of zero values). Google says that for the Flu, the average person with the disease spreads it to 1-2 people. As our negative binomial distribution only counts the number of additional people each person infects after the first, then setting $E[X] = 0.5$ feels fair. In other words, for the flu, it reasonable to stay the average person in a hospital with a disease infects 0.5 extra people. That would means that $\mathbb{E}[X] = \frac{r(1-p)}{p} = 0.5$.

I isolated for p in Homework 6, and got that $p = 0.6$:

Let's say our null hypothesis is $H_0: p = 0.6$. Now, we are curious if maybe COVID has a different degree of infectiousness than the flu (not considering other considerations like how my dataset is limited to people only already at the hospital). This would mean that $E[X] \neq 0.5$, and $p \neq 0.6$.

So, this gives us:

$$H_0: p = 0.6$$

$$H_A: p \neq 0.6$$

Now, for Bayesian Hypothesis Testing, we need to assign prior probabilities to both the null hypothesis and the alternative hypothesis. As I do not really have a great intuition, let's say both are 0.5.

So, we have:

$$P(H_0) = 0.5$$

$$P(H_A) = 0.5$$

Next, we need to find the prior distribution for p in H_A . Again, as I do not really have a great intuition, let's say $p \sim \text{Uniform}(0, 1)$, as p could be anything from 0 to 1.

So, using Bayes' Rule, this means that:

$$P(H_A|x) = \frac{P(H_A)f(x|H_A)}{f(x)}$$

$$P(H_0|x) = \frac{P(H_0)f(x|H_0)}{f(x)}$$

NOTE: In the formula below, I will refer to p in H_0 as p_0 . I know we set this equal to 0.6, but for clarity, I think it is easier to understand with p_0 .

Remember that $f(x_1, x_2, \dots, x_n)$ is the product of Negative Binomial random variables.

When taking their ratio, we get the Bayes Factor, which is:

$$\begin{aligned} \frac{P(H_A|x_1, \dots, x_n)}{P(H_0|x_1, \dots, x_n)} &= \frac{\frac{P(H_A)f(x_1, \dots, x_n|H_A)}{f(x_1, \dots, x_n)}}{\frac{P(H_0)f(x_1, \dots, x_n|H_0)}{f(x_1, \dots, x_n)}} \\ &= \frac{P(H_A)f(x_1, \dots, x_n|H_A)}{P(H_0)f(x_1, \dots, x_n|H_0)} \\ &= \frac{P(H_A)}{P(H_0)} \frac{f(x_1, \dots, x_n|H_A)}{f(x_1, \dots, x_n|H_0)} \\ &= \frac{0.5}{0.5} \frac{\int_0^1 f(p)f(x_1, \dots, x_n|p)dp}{\prod_{i=1}^n \binom{x_i+r-1}{x_i} (p_0)^r (1-p_0)^{x_i}} \\ &= \frac{\int_0^1 f(x_1, \dots, x_n|p)dp}{\prod_{i=1}^n \binom{x_i+r-1}{x_i} (p_0)^r (1-p_0)^{x_i}} \\ &= \frac{\int_0^1 \prod_{i=1}^n \binom{x_i+r-1}{x_i} p^r (1-p)^{x_i} dp}{\prod_{i=1}^n \binom{x_i+r-1}{x_i} (p_0)^r (1-p_0)^{x_i}} \\ &= \frac{\int_0^1 \prod_{i=1}^n \binom{x_i+r-1}{x_i} p^r (1-p)^{x_i} dp}{\prod_{i=1}^n \binom{x_i+r-1}{x_i} (p_0)^r (1-p_0)^{x_i}} \\ &= \frac{\int_0^1 \left(\prod_{i=1}^n \binom{x_i+r-1}{x_i} \right) p^{rn} (1-p)^{\sum_{i=1}^n x_i} dp}{\left(\prod_{i=1}^n \binom{x_i+r-1}{x_i} \right) (p_0)^{rn} (1-p_0)^{\sum_{i=1}^n x_i}} \\ &= \frac{\left(\prod_{i=1}^n \binom{x_i+r-1}{x_i} \right) \int_0^1 p^{rn} (1-p)^{\sum_{i=1}^n x_i} dp}{\left(\prod_{i=1}^n \binom{x_i+r-1}{x_i} \right) (p_0)^{rn} (1-p_0)^{\sum_{i=1}^n x_i}} \\ &= \frac{\int_0^1 p^{rn} (1-p)^{\sum_{i=1}^n x_i} dp}{(p_0)^{rn} (1-p_0)^{\sum_{i=1}^n x_i}} \end{aligned}$$

We can simplify the numerator by recognizing that it has the meat of the Beta PDF over its domain.

$$\begin{aligned}
\frac{P(H_A|x)}{P(H_0|x)} &= \frac{\int_0^1 p^{rn}(1-p)^{\sum_{i=1}^n x_i} dp}{(p_0)^{rn}(1-p_0)^{\sum_{i=1}^n x_i}} \\
&= \frac{\frac{\Gamma(rn+1)\Gamma(\sum_{i=1}^n x_i+1)}{\Gamma(rn+1+\sum_{i=1}^n x_i+1)} \int_0^1 \frac{\Gamma(rn+1+\sum_{i=1}^n x_i+1)}{\Gamma(rn+1)\Gamma(\sum_{i=1}^n x_i+1)} p^{rn}(1-p)^{\sum_{i=1}^n x_i} dp}{(p_0)^{rn}(1-p_0)^{\sum_{i=1}^n x_i}} \\
&= \frac{\frac{\Gamma(rn+1)\Gamma(\sum_{i=1}^n x_i+1)}{\Gamma(rn+\sum_{i=1}^n x_i+2)}}{(p_0)^{rn}(1-p_0)^{\sum_{i=1}^n x_i}}
\end{aligned}$$

In my hospital dataset, the data seems to fit a negative binomial distribution in which r is not necessarily an integer, meaning we cannot simplify this further with factorials.

Instead, let's plug our data this equation to find the Bayes Factor. Let's say $r = 0.75$, the same r value I used in Homework 7.

Unfortunately, the denominator gets so small that the code thinks we are dividing by 0. So, let's instead take e to the power of [the log of the numerator subtracted by the $\log$ of the denominator].

$$\begin{aligned}
\frac{P(H_A|x)}{P(H_0|x)} &= \frac{\frac{\Gamma(rn+1)\Gamma(\sum_{i=1}^n x_i+1)}{\Gamma(rn+\sum_{i=1}^n x_i+2)}}{(p_0)^{rn}(1-p_0)^{\sum_{i=1}^n x_i}} \\
&= e^{\log\left(\frac{\Gamma(rn+1)\Gamma(\sum_{i=1}^n x_i+1)}{\Gamma(rn+\sum_{i=1}^n x_i+2)}\right) - \log\left((p_0)^{rn}(1-p_0)^{\sum_{i=1}^n x_i}\right)} \\
&= e^{\log\left(\frac{\Gamma(rn+1)\Gamma(\sum_{i=1}^n x_i+1)}{\Gamma(rn+\sum_{i=1}^n x_i+2)}\right) - \log\left((p_0)^{rn}(1-p_0)^{\sum_{i=1}^n x_i}\right)} \\
&= e^{\log\left(\Gamma(rn+1)\right) + \log\left(\Gamma(\sum_{i=1}^n x_i+1)\right) - \log\left(\Gamma(rn+\sum_{i=1}^n x_i+2)\right) - \left(rn \log(p_0) + \sum_{i=1}^n x_i \log(1-p_0)\right)}
\end{aligned}$$

```

r = 0.75
sum_x = sum(x)
n = length(x)
p_0 = 0.6

first_term = lgamma(r * n+1)
second_term = lgamma(sum_x + 1)
third_term = lgamma(r * n + sum_x + 2)

```

```
fourth_term = r * n * log(p_0) + sum_x * log(1-p_0)

exponent = first_term + second_term - third_term - fourth_term

bayes_factor = exp(exponent)
bayes_factor
```

My Bayes Factor is 607815364, meaning there is very strong evidence to reject the null hypothesis per the criteria in Kass and Raftery (1995).

In Homework 7, using Frequentist Hypothesis Testing methods, I also rejected the null hypothesis. So, both methods reached the same conclusion, in that we rejected H_0 (we reject the hypothesis that COVID has the same contagious level as the flu). This means that I did not encounter Lindley's paradox.

10 Sufficiency

10.1 Sufficient Statistic

In class, we learned that for data $X_1, \dots, X_n$ and parameter θ , a statistic $T(x_1, \dots, x_n)$ is sufficient if either of the following hold: \

1: $f_{\theta}(x_1, \dots, x_n) = g(\theta, T(x_1, \dots, x_n)) \cdot h(x_1, \dots, x_n),$

where:

$g(\theta, T(x_1, \dots, x_n))$ depends on θ only through T

$h(x_1, \dots, x_n)$ depends only on $(x_1, \dots, x_n)$ and not on θ

2. $P(X_1 = x_1, \dots, X_n = x_n | T = t)$ does not depend on θ .

For my Negative Binomial dataset, p is the parameter we are interested in estimating, so p will be our θ .

I will show that for my distribution, both definitions hold:

10.1.1 First Proof

The PDF of my negative Binomial Distribution: $X_1, \dots, X_n \sim \text{NegBin}(r, p)$ is:

$$f_p(x_1, \dots, x_n) = \prod_{i=1}^n \binom{x_i + r - 1}{x_i} p^r (1 - p)^{x_i}$$

We can simplify this into:

$$\begin{aligned}
f_p(x_1, \dots, x_n) &= \prod_{i=1}^n \binom{x_i + r - 1}{x_i} p^r (1-p)^{x_i} \\
&= p^{rn} \prod_{i=1}^n \binom{x_i + r - 1}{x_i} (1-p)^{x_i} \\
&= p^{rn} \left(\prod_{i=1}^n \binom{x_i + r - 1}{x_i} \right) (1-p)^{\sum_{i=1}^n x_i}
\end{aligned}$$

We can re-write this as:

$$f_p(x_1, \dots, x_n) = p^{rn} (1-p)^{\sum_{i=1}^n x_i} \cdot \prod_{i=1}^n \binom{x_i + r - 1}{x_i}$$

Now, as we have been doing, let's assume r is fixed and p is the parameter we are interested in estimating.

Here, if we set $T(X_1, \dots, x_n) = \sum_{i=1}^n x_i$, then we can rewrite the equation as:

$$\begin{aligned}
f_p(x_1, \dots, x_n) &= p^{rn} (1-p)^{\sum_{i=1}^n x_i} \cdot \prod_{i=1}^n \binom{x_i + r - 1}{x_i} \\
&= p^{rn} (1-p)^T \cdot \prod_{i=1}^n \binom{x_i + r - 1}{x_i} \\
&= g(p, T(x_1, \dots, x_n)) \cdot h(x_1, \dots, x_n)
\end{aligned}$$

where:

1. $g(p, T(x_1, \dots, x_n)) = p^{rn} (1-p)^T$
2. $h(x_1, \dots, x_n) = \prod_{i=1}^n \binom{x_i + r - 1}{x_i}$

As we were able to write our PDF in terms of $g(p, T(x_1, \dots, x_n)) \cdot h(x_1, \dots, x_n)$, then this means $T(x_1, \dots, x_n)$ is a sufficient statistic.

So, for the Negative Binomial distribution, $\sum_{i=1}^n x_i$ is a sufficient statistic.

10.1.2 Second Proof

Similarly, we could have also confirmed this using the other definition we learned in class, that $P(X_1 = x_1, \dots, X_n = x_n | T = t)$ does not depend on p .

$$P(X_1 = x_1, \dots, X_n = x_n | T = t) = \frac{P(X_1 = x_1, \dots, X_n = x_n, T = t)}{P(T = t)}$$

T is the sum of i.i.d. $\text{NegBin}(r, p)$ random variables. So, T is a $\text{NegBin}(nr, p)$ random variable.

So, we get:

$$\begin{aligned} P(X_1 = x_1, \dots, X_n = x_n | T = t) &= \frac{P(X_1 = x_1, \dots, X_n = x_n, T = t)}{P(T = t)} \\ P(X_1 = x_1, \dots, X_n = x_n | T = t) &= \frac{p^{rn}(1-p)^t \cdot \prod_{i=1}^n \binom{x_i+r-1}{x_i}}{\binom{t+nr-1}{t} p^{rn}(1-p)^t} \\ P(X_1 = x_1, \dots, X_n = x_n | T = t) &= \frac{\prod_{i=1}^n \binom{x_i+r-1}{x_i}}{\binom{t+nr-1}{t}} \end{aligned}$$

As we can see, this does not depend on p , proving once again that $T(X_1, \dots, x_n) = \sum_{i=1}^n x_i$ is a sufficient statistic.

References

Knuth, Donald E. 1984. “Literate Programming.” *Comput. J.* (USA) 27 (2): 97–111.
<https://doi.org/10.1093/comjnl/27.2.97>.